

BotLaunch — Whitepaper

Token launch infrastructure for BOT Chain — clear the gate, then get a pool.

Version 1 · Testnet 968 · September 2026

Abstract

BotLaunch is developer tooling that turns token issuance on BOT Chain into a guarded, two-step flow. A factory contract mints a plain ERC-20 into the creator's wallet. An audited, V3-compatible pool engine creates the liquidity pair on request. And a third-party gatekeeper — never the token creator — approves each listing on-chain, on a two-key wall the creator does not control.

Until a token is verified, the pool engine refuses to list it with a `NotVerified` revert. Verified or not, the token's supply lands in the creator's wallet, its liquidity stays in the creator's hands, and every gate decision is stamped on chain. The result is a chain-of-custody signal for memecoins: honest market intent, a reviewable pedigree, and no silent rugs on audited DEX rails.

The problem: token launch is structurally broken

Most memecoin launches fail for reasons that are entirely foreseeable:

- **Clown taxes.** The contract subtracts 20–30% on every sale, so the DEX router cannot price it. The tax exists to be seen, not for any product reason.
- **Honeypots.** The creator can block sells outright, trapping everyone who bought in.
- **Unlocked LP.** The creator holds the LP keys and can pull liquidity the moment price is interesting.
- **Renounce ceremonies.** The creator "renounces" ownership on a contract where renouncement was always cosmetic, then finds a backdoor later.

- **No chain of custody.** None of this is visible at listing time. Buyers get a default "sounds fine, trust me" experience.

Audits help, but audits are a point-in-time opinion. The market needs a **gate that stays armed between deployments.**

The gate pattern

BotLaunch splits issuance from listing responsibility:

1. **Anyone can mint.** `createToken(name, symbol, supply)` deploys a plain, taxless ERC-20 minted to `tx.from`. No ceremony, no permission, no hidden hooks.
2. **Listing is gated.** `ensurePool(token, base, fee)` pulls or creates the pool on the audited V3 engine — but *only* if a gatekeeper operator has already flipped `setVerified(token, true)`. Otherwise it reverts with `NotVerified`.
3. **Verification is a business decision, not a technical one.** The gatekeeper reviews the token, the team, and the market intent, then stamps the ledger. The stamp is the signal; the contract is the enforcement.

The gatekeeper can also be switched off at the operator level with `setGating(false)`, which re-opens listing for everything — the intended escape hatch for a rule-proofed future, not a daily knob.

Architecture

Component	Responsibility
<code>TokenFactory</code>	Deploys plain, taxless ERC-20s minted to the creator.
<code>PoolEngine</code>	Audited V3-compatible DEX engine; creates and reads back pools.

Component	Responsibility
Gatekeeper	Two-key wall: owner + an independent gatekeeper operator. <code>setVerified</code> stamps the ledger; <code>setGating</code> arms or opens the gate.

Issuance flow

```

createToken(name, symbol, supply)
  ↳ ERC-20 lands in creator wallet (full supply, tx.from)

gatekeeper review → setVerified(token, true)
  ↳ ledger stamped "cleared"

ensurePool(token, base, fee)
  ↳ gate OFF → pool created immediately
  ↳ gate ON → pool created only after verified
               else revert NotVerified

creator adds liquidity (own keys, own LP)

```

Only 2–3 transactions separate a raw idea from a listed, pooled token — every one of them visible on the ledger.

Security model

- **Two-key wall.** Listing authority sits with a gatekeeper independent of minting authority. One compromised key cannot both mint and list.
- `NotVerified` is an **on-chain revert**, not a UI warning. Pooling simply cannot happen until the stamp is on the ledger.
- **Adversarial contracts are flagged on review** — clown taxes, honeypots, LP locks without a timelock — and kept out of the ledger.
- **Liquidity is the creator's.** The pool is created; the LP is added and held by the creator. BotLaunch never touches it.
- **The gate is a chain-of-custody signal**, not a legal or financial opinion. "Cleared" means the gatekeeper verified honest intent, not that a token will

go up.

Roadmap

Item	Status
Listing gate + verified readback	Shipped (testnet 968)
Liquidity locker — time-locked LP separate from the router path, so rug pressure can never close a listed pool	Live (testnet 968, LiquidityLocker 0x9276644dC1E26a6d183a5e76321BF6e92a0c2d67)
Two-key signing split — owner holds gating + roles, gatekeeper alone signs list/admit; neither key does both	Live (testnet 968)
Pedigree explorer — who minted, which pools cleared, LP positions live	Parked
Metadata registry — on-chain URI per token for dApps/aggregators	Parked
Sandbox mainnet — full starting-block	Live

Item	Status
copy of BOT Chain for micro-launches	
Base and BOT rails — one factory, one gate wrapper per chain	Next (no Base deployment yet)

No token

BotLaunch is tooling, not a token. There is no allocation, no tax, no renounce ceremony to excuse. The product is the gate, and the gate is free to audit.

Disclaimer

This document describes developer tooling on BOT Chain testnet (968) and BOT-V3-compatible DEX rails. Gate status is a chain-of-custody signal, not investment advice. Past clearances do not predict performance; liquidity is yours to add; nothing here promises yields, price, or protection from your own market. DYOR.